
3. WAP to implement a three-layer neural network using Tensorflow library (no keras) to classify MNIST handwritten digits dataset. Demonstrate the implementation of feed-forward and back-propagation approaches.

✓ Explanation of the Code

This code implements a three-layer neural network (MLP) using TensorFlow (without Keras) to classify the MNIST handwritten digits dataset. It uses feed-forward and backpropagation to train the model.

✓ 1. Importing Libraries

```
import tensorflow as tf
import tensorflow_datasets as tfds
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
from sklearn.metrics import confusion_matrix
```

- **TensorFlow:** Used to build the neural network.
 - **tensorflow_datasets (tfds):** Fetches the MNIST dataset.
 - **numpy:** For array manipulations.
 - **matplotlib & seaborn:** Used for visualization.
 - **confusion_matrix:** Evaluates classification performance.
-

✓ 2. Disabling Eager Execution

```
# Disable eager execution
tf.compat.v1.disable_eager_execution()
```

 WARNING:tensorflow:From C:\Users\harsh\AppData\Local\Temp\ipykernel_10364\1016160911.py:2: The name tf.disable_eager_execution is deprecated. Please use tf.compat

- Disables **eager execution** to use **TensorFlow 1.x-style** graph execution.
-

✓ 3. Loading and Preparing MNIST Dataset

```
# Load MNIST dataset
mnist, info = tfds.load('mnist', with_info=True, as_supervised=True)
```

- Loads the **MNIST dataset**.
 - **as_supervised=True** ensures it returns (image, label) pairs.
-

```
# Parameters
learning_rate = 0.01
num_steps = 1000
batch_size = 128
display_step = 100

# Network Parameters
n_hidden_1 = 256 # 1st layer number of neurons
n_hidden_2 = 256 # 2nd layer number of neurons
n_input = 784 # MNIST data input (img shape: 28*28)
n_classes = 10 # MNIST total classes (0-9 digits)
```

Preprocessing Function

```
# Prepare the data
def preprocess(image, label):
    image = tf.reshape(image, [n_input])
    image = tf.cast(image, tf.float32) / 255.0
    label = tf.one_hot(label, n_classes)
    return image, label
```

- **Reshapes** the 28×28 image into a 784-dimensional vector.
 - **Normalizes** pixel values to [0,1].
 - **One-hot encodes** labels (e.g., 3 → [0,0,0,1,0,0,0,0,0,0]).
-

```
train_data = mnist['train'].map(preprocess).shuffle(60000).batch(batch_size)
test_data = mnist['test'].map(preprocess).batch(batch_size)
```

- **Shuffles** and **batches** the dataset.
-

✓ 4. Creating Placeholders

```
# Create an iterator
iterator = tf.compat.v1.data.make_initializable_iterator(train_data)
next_element = iterator.get_next()
```

```
# tf Graph input
X = tf.compat.v1.placeholder("float", [None, n_input])
Y = tf.compat.v1.placeholder("float", [None, n_classes])
```

⚠ WARNING:tensorflow:From C:\Users\harsh\AppData\Local\Temp\ipykernel_10364\1093387182.py:2: The name tf.data.make_initializable_iterator is deprecated. Please use

WARNING:tensorflow:From C:\Users\harsh\AppData\Local\Temp\ipykernel_10364\1093387182.py:2: The name tf.data.make_initializable_iterator is deprecated. Please use

- x: Placeholder for input images (**batch_size, 784**).
- y: Placeholder for labels (**batch_size, 10**).

5. Initializing Weights and Biases

```
# Store layers weight & bias
weights = {
    'h1': tf.Variable(tf.random.normal([n_input, n_hidden_1])),
    'h2': tf.Variable(tf.random.normal([n_hidden_1, n_hidden_2])),
    'out': tf.Variable(tf.random.normal([n_hidden_2, n_classes]))
}
biases = {
    'b1': tf.Variable(tf.random.normal([n_hidden_1])),
    'b2': tf.Variable(tf.random.normal([n_hidden_2])),
    'out': tf.Variable(tf.random.normal([n_classes]))
}
```

- Randomly initializes **weights** and **biases** for:
 - **Layer 1:** 784 → 256
 - **Layer 2:** 256 → 256
 - **Output Layer:** 256 → 10

6. Neural Network Model (Feed-Forward)

```
# Create model
def neural_net(x):
    # Hidden fully connected layer with 256 neurons
    layer_1 = tf.add(tf.matmul(x, weights['h1']), biases['b1'])
    # Apply ReLU activation
    layer_1 = tf.nn.relu(layer_1)

    # Hidden fully connected layer with 256 neurons
    layer_2 = tf.add(tf.matmul(layer_1, weights['h2']), biases['b2'])
    # Apply ReLU activation
    layer_2 = tf.nn.relu(layer_2)

    # Output fully connected layer with a neuron for each class
    out_layer = tf.matmul(layer_2, weights['out']) + biases['out']
    return out_layer
```

- Uses **ReLU activation** ($f(x) = \max(0, x)$) for hidden layers.
- The **output layer** does not use an activation function.

7. Loss and Optimization (Backpropagation)

```
# Construct model
logits = neural_net(X)

# Define loss and optimizer
loss_op = tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits(logits=logits, labels=Y))
optimizer = tf.compat.v1.train.AdamOptimizer(learning_rate=learning_rate)
train_op = optimizer.minimize(loss_op)
```

⚠ WARNING:tensorflow:From C:\Users\harsh\AppData\Local\Temp\ipykernel_10364\146740076.py:6: The name tf.train.AdamOptimizer is deprecated. Please use tf.compat.v1.t

WARNING:tensorflow:From C:\Users\harsh\AppData\Local\Temp\ipykernel_10364\146740076.py:6: The name tf.train.AdamOptimizer is deprecated. Please use tf.compat.v1.t

- **Softmax Cross-Entropy** loss for classification.
- **Adam Optimizer** updates weights via backpropagation.

8. Model Evaluation

```
# Evaluate model
correct_pred = tf.equal(tf.argmax(logits, 1), tf.argmax(Y, 1))
accuracy = tf.reduce_mean(tf.cast(correct_pred, tf.float32))
```

- **Correct Predictions:** Compares predicted vs actual labels.
- **Accuracy:** Computes mean classification accuracy.

9. Training the Model

- **Fetches batches** from dataset.
- **Performs backpropagation** to update weights.

- **Displays loss & accuracy** every 100 steps.

```
# Initialize the variables (i.e. assign their default value)
init = tf.compat.v1.global_variables_initializer()

# Lists for visualization
loss_history = []
accuracy_history = []

# Start training
with tf.compat.v1.Session() as sess:
    # Run the initializer
    sess.run(init)
    sess.run(iterator.initializer)

    for step in range(1, num_steps + 1):
        try:
            batch_x, batch_y = sess.run(next_element)
        except tf.errors.OutOfRangeError:
            sess.run(iterator.initializer)
            batch_x, batch_y = sess.run(next_element)
        # Run optimization op (backprop)
        _, loss, acc = sess.run([train_op, loss_op, accuracy], feed_dict={X: batch_x, Y:
            batch_y})
        loss_history.append(loss)
        accuracy_history.append(acc)

        if step % display_step == 0 or step == 1:
            print("Step {}: Minibatch Loss = {:.4f}, Training Accuracy = {:.3f}".format
                (step, loss, acc))

    print("Optimization Finished!")

#
# *****
# *****

# ***** 10. Testing the Model *****

# Calculate accuracy for MNIST test images
test_iterator = tf.compat.v1.data.make_initializable_iterator(test_data)
next_test_element = test_iterator.get_next()
sess.run(test_iterator.initializer)
test_acc = 0
test_count = 0
y_true, y_pred = [], []

while True:
    try:
        test_images, test_labels = sess.run(next_test_element)
        acc, preds = sess.run([accuracy, tf.argmax(logits, 1)], feed_dict={X:
            test_images, Y: test_labels})
        y_true.extend(np.argmax(test_labels, axis=1))
        y_pred.extend(preds)
        test_acc += acc
        test_count += 1
    except tf.errors.OutOfRangeError:
        break
test_acc /= test_count
print("Testing Accuracy:", test_acc)

# ***** Iterates over the test dataset and calculates accuracy. *****

#
# *****

# ***** 11. Visualization *****

# ***** Plot Loss Curve *****

plt.figure()
plt.plot(loss_history, label='Loss')
plt.xlabel('Steps')
plt.ylabel('Loss')
plt.title('Loss Curve')
plt.legend()
plt.show()

# ***** Plots the loss curve over training. *****

# ***** Plot Accuracy Curve *****

plt.figure()
plt.plot(accuracy_history, label='Accuracy')
plt.xlabel('Steps')
plt.ylabel('Accuracy')
plt.title('Accuracy Curve')
plt.legend()
plt.show()

# ***** Plots accuracy improvement over time. *****

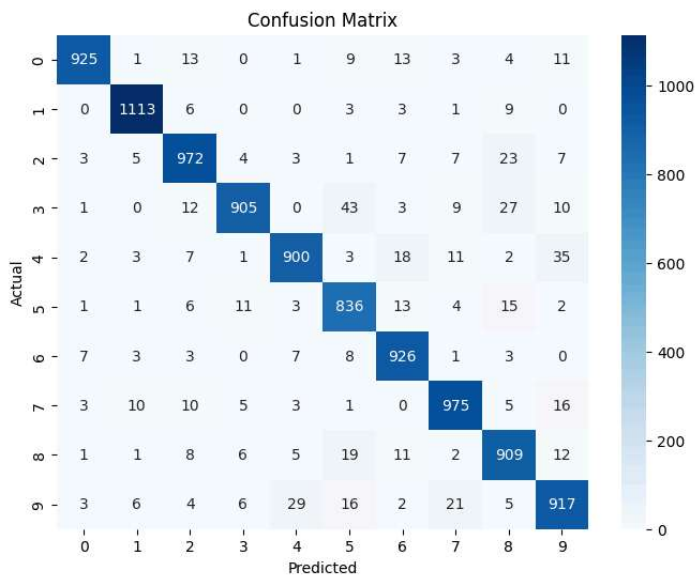
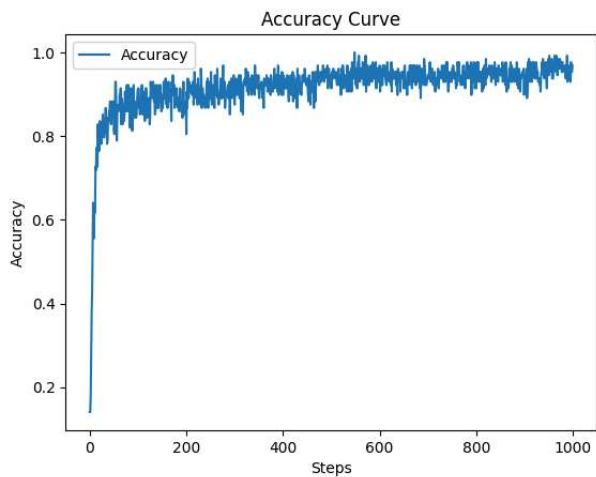
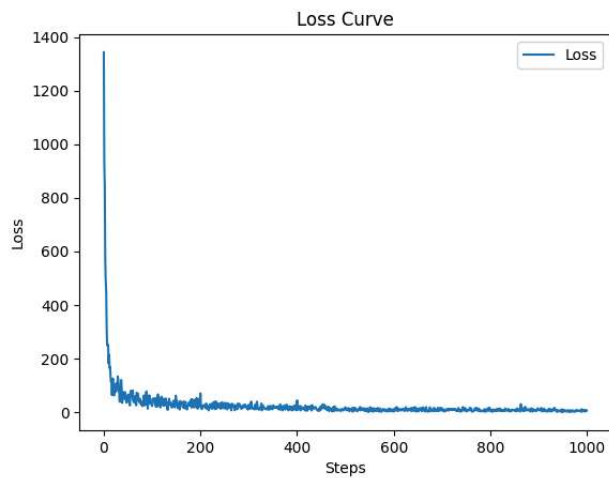
# ***** Plot Confusion Matrix *****

cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=range(10),
    yticklabels=range(10))
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
plt.show()

# ***** Confusion Matrix shows classification performance *****
```

***** Confusion Matrix Shows Classification performance. *****

Step 1: Minibatch Loss = 1342.7188, Training Accuracy = 0.141
Step 100: Minibatch Loss = 32.4566, Training Accuracy = 0.906
Step 200: Minibatch Loss = 28.4299, Training Accuracy = 0.883
Step 300: Minibatch Loss = 24.7994, Training Accuracy = 0.891
Step 400: Minibatch Loss = 9.3233, Training Accuracy = 0.930
Step 500: Minibatch Loss = 11.9424, Training Accuracy = 0.945
Step 600: Minibatch Loss = 10.7255, Training Accuracy = 0.930
Step 700: Minibatch Loss = 14.6834, Training Accuracy = 0.938
Step 800: Minibatch Loss = 2.7554, Training Accuracy = 0.969
Step 900: Minibatch Loss = 9.9673, Training Accuracy = 0.898
Step 1000: Minibatch Loss = 4.3983, Training Accuracy = 0.969
Optimization Finished!
Testing Accuracy: 0.9384889



Summary

- ☒ Implements **MLP** with **TensorFlow** (without Keras)
- ☒ Uses **feed-forward & backpropagation**
- ☒ **Trains & evaluates** on MNIST dataset